Guion del video — Semana 9

Nov 7, 2026 · @Jhordi Chauca Machuca

Resumen

Un solo video de 9 minutos cubre las dos entregas: la exposición de TypeScript (8 a 10 minutos) y la demostración del laboratorio de Motion. Si el docente pide videos separados, las partes de Patiño, Yvette y la primera mitad de Jhordi son TypeScript; el resto es Motion.

Integrante	Tiempo	Tema
Patiño	0:00 – 1:45	Proyecto, problema y plan
Yvette	1:45 – 4:00	Tipos, props y ExperienceCard
Jhordi	4:00 – 6:30	Favoritos y animación con Motion
Tesen	6:30 – 9:00	Pruebas, accesibilidad y cierre

Preparación antes de grabar

Dejen todo abierto antes de empezar, para que ninguna demostración consuma tiempo del video.

1. Ejecuten `npm install` y `npm run dev`, y abran `/explorador-cursos` en Chrome.
2. Borren los favoritos de prueba: en DevTools, Application, Local Storage, eliminen `wayra-favoritos`.
3. Dejen abiertos en el editor `src/types/experience.ts`, `src/datos/cursos.ts`, `src/componentes/ExperienceCard.tsx`, `src/hooks/useFavoritos.ts` y `src/animaciones/tarjeta.ts`.
4. Dejen una terminal lista en la carpeta del proyecto.
5. Suban el zoom del editor y del navegador para que el texto se lea en el video.
6. Cierren notificaciones y pestañas personales.

Patiño — Proyecto y problema (0:00 – 1:45)

Tiempo	En pantalla	Qué decir
0:00	Portada de Wayra Academy	Hola, somos Patiño, Yvette, Jhordi y Tesen. Les presentamos el avance de la semana 9 de Wayra Academy, nuestra plataforma de cursos en línea. Esta semana trabajamos dos cosas sobre un mismo componente: TypeScript y animación con Motion.
0:25	Abrir el explorador de cursos	Este es el explorador. Aquí cada curso se muestra en una tarjeta, y en nuestro proyecto cada curso es una experiencia de la plataforma. Hay 16 cursos, con filtros y búsqueda.
0:50	Una tarjeta de cerca	El problema era este: en JavaScript, nada impedía que a una tarjeta le faltara un dato, o que recibiera una valoración como texto. El error se veía recién en el navegador, y a veces ni siquiera se notaba.
1:15	Seguir en la tarjeta	Por eso nos propusimos dos objetivos. Primero, que los datos sean confiables, con contratos claros. Segundo, que la interfaz responda mejor a cada acción, sin perder accesibilidad ni rendimiento.
1:35	Editor con `experience.ts`	Migramos de forma progresiva, sin cambiar toda la aplicación de golpe. Yvette va a explicar cómo definimos esos contratos.

Yvette — Tipos, props y ExperienceCard (1:45 – 4:00)

Tiempo	En pantalla	Qué decir
1:45	`src/types/experience.ts`	Este archivo es el contrato de datos. La interfaz `Experience` define qué tiene un curso: título, texto, categoría, nivel, horas, valoración y temario. Está en un solo lugar, así que no repetimos tipos.
2:10	Señalar `ExperienceCategory` y `imagenUrl?`	La categoría es una unión de cuatro valores, así que no se acepta otra. El signo de interrogación de `imagenUrl` indica que el campo es opcional.
2:30	`src/datos/cursos.ts`: borrar `valoracion` de un curso, sin guardar	Ahora veamos un error real. Si le quito la valoración a este curso, el editor lo marca enseguida. TypeScript avisa antes de ejecutar.
2:50	Cambiar una categoría a `"disenio"`, sin guardar	Y si escribo mal una categoría, también falla. Antes esto habría dejado una etiqueta vacía, sin ningún aviso. Deshago los cambios.
3:05	`ExperienceCard.tsx`, en `ExperienceCardProps`	Esta es la tarjeta. Sus props también están tipadas: `item` es obligatorio, y `onSelect` y el favorito son opcionales. Si el padre no pasa la función, el botón simplemente no aparece.
3:25	Navegador: tarjeta de Frontend junto a la de React	Una sola tarjeta sirve para los 16 cursos. Frontend tiene imagen. React no la tiene, así que muestra nuestra chakana como alternativa visual.

Tiempo	En pantalla	Qué decir
3:45	Terminal: `npm run typecheck` sin errores	Como Vite no revisa los tipos al compilar, agregamos esta comprobación al build. El proyecto compila sin errores y sin usar `any`. Jhordi sigue con lo que pasa al interactuar.

Jhordi — Favoritos y animación con Motion (4:00 – 6:30)

Tiempo	En pantalla	Qué decir
4:00	Navegador: pulsar «Agregar a favoritos»	Al pulsar, el botón cambia su etiqueta a «Quitar de favoritos» y su estado `aria-pressed`, que es lo que anuncia un lector de pantalla.
4:15	Filtrar por otra categoría, limpiar y recargar	Filtro, vuelvo y recargo la página: el favorito sigue ahí. Los filtros desmontan las tarjetas, así que un estado dentro de la tarjeta se habría perdido.
4:40	`src/hooks/useFavoritos.ts`	Por eso creamos este custom hook, que guarda los favoritos en `localStorage`. Lo que se lee de ahí se valida al ejecutar, porque TypeScript no controla datos externos.
5:00	Recargar y mirar la entrada de las tarjetas	Pasamos a Motion. Al cargar, las tarjetas entran con un fundido de unos 300 milisegundos. Es corto a propósito: orienta sin retrasar la lectura.
5:20	Bajar por el catálogo y volver a subir	Cada tarjeta aparece al llegar a la pantalla, y solo una vez. Al volver a subir no se repite, para no distraer.
5:35	Pasar el mouse sobre una tarjeta	Con el puntero, la tarjeta sube 3 píxeles. No usamos escala para que el texto no se vea borroso.

Tiempo	En pantalla	Qué decir
5:50	Pulsar «Ver experiencia» y luego el favorito	Al pulsar, el botón se reduce un instante y se abre el temario. Al marcar un favorito, la estrella hace un pulso breve. Esa es nuestra microinteracción: confirma la acción.
6:10	`src/animaciones/tarjeta.ts`	Todos estos estados están en un solo archivo, con variants tipadas. Descartamos el arrastre porque no aportaba nada al catálogo y podía estorbar el desplazamiento en móvil. Tesen explica cómo lo probamos.

Tesen — Pruebas, accesibilidad y cierre (6:30 – 9:00)

Tiempo	En pantalla	Qué decir
6:30	Navegador: recorrer con Tab	Primero, el teclado. Con Tab llego a cada control y el foco siempre se ve. La tarjeta también se marca cuando el foco está dentro.
6:50	Pulsar Enter en el favorito y en «Ver experiencia»	Con Enter activo los botones igual que con el mouse. Las animaciones no quitan ninguna función.
7:05	DevTools, Rendering: activar `prefers-reduced-motion: reduce` y recargar	Ahora simulo la preferencia de reducir movimiento. Las tarjetas se muestran de inmediato, y no hay desplazamiento en hover ni al pulsar. El favorito sigue funcionando.
7:30	Quitar la simulación y abrir la vista móvil de DevTools, a 375 píxeles	En móvil no hay desbordamiento y el desplazamiento funciona también sobre las tarjetas.

Tiempo	En pantalla	Qué decir
7:45	Tocar el favorito en la vista móvil	Aquí encontramos un error revisando las capturas. Tras el toque, un botón ya desmarcado seguía azul, porque el hover se quedaba pegado. Lo corregimos: el hover solo aplica con puntero real.
8:05	Tabla de resultados en `docs/LABORATORIO-MOTION.md`	Las 8 pruebas del protocolo pasaron. axe no encontró violaciones en 20 combinaciones. Lighthouse dio 97 de rendimiento en móvil, 100 en escritorio y 100 en accesibilidad.
8:25	Tabla de peso del mismo documento	También cuidamos el peso. Motion completo aumentaba el JavaScript en 47 %. Cargando solo lo necesario, y en un archivo aparte, el principal sube 14 %.
8:40	Volver al explorador	En resumen: TypeScript nos da datos confiables, y Motion, una respuesta clara a cada acción. TypeScript no reemplaza las pruebas: el error del hover pasó la compilación. Nos falta probar en un teléfono físico. Gracias por su atención.

Lista de control final

Los tiempos son una estimación: hagan un ensayo cronometrado y ajusten antes de grabar. Revisen esto antes de entregar el video.

- [ ] Dura entre 8 y 10 minutos
- [ ] Se ve la entrada de las tarjetas
- [ ] Se ven el hover y el tap
- [ ] Se ve la vista móvil y de escritorio
- [ ] Se ve la prueba con teclado
- [ ] Se ve la prueba de movimiento reducido

- [ ] Los cambios de prueba en `cursos.ts` quedaron deshechos
- [ ] Los cuatro integrantes participan y se les escucha bien